

AssignmentNumber : 12.4

Q.No.	Question
1	Lab 12 – Algorithms with AI Assistance – Sorting, Searching, and Optimizing Algorithms Lab Objectives • Apply AI-assisted programming to implement and optimize sorting and searching algorithms. • Compare different algorithms in terms of efficiency and use cases. • Understand how AI tools can suggest optimized code and complexity improvements. Learning Outcome After completing this assignment, students will be able to: • Implement classical algorithms with AI assistance • Compare algorithm efficiency using real-world scenarios • Understand when optimization is necessary • Critically evaluate AI-generated suggestions instead of blindly accepting them
	Task 1: Bubble Sort for Ranking Exam Scores Scenario You are working on a college result processing system where a small list of student scores needs to be sorted after every internal assessment. Task Description • Implement Bubble Sort in Python to sort a list of student scores. • Use an AI tool to: ○ Insert inline comments explaining key operations such as comparisons, swaps, and iteration passes ○ Identify early-termination conditions when the list becomes sorted ○ Provide a brief time complexity analysis Expected Outcome

- A Bubble Sort implementation with:
 - AI-generated comments explaining the logic
 - Clear explanation of best, average, and worst-case complexity
 - Sample input/output showing sorted scores

Prompt :

Implement Bubble Sort in Python to rank student exam scores for a small dataset. Add inline comments explaining comparisons, swaps, and iteration passes. Include an early-termination condition when the list is already sorted. Provide a brief analysis of best, average, and worst-case time complexity, and show sample input/output with sorted scores.

Certainly! I will generate the Python code for the Bubble Sort algorithm.

Code :

```
def bubble_sort(scores):
    n = len(scores)
    # Traverse through all elements in the list
    for i in range(n):
        swapped = False # Flag to optimize by detecting if any swaps occurred in this pass

        # Last i elements are already in place, so we don't need to check them
        for j in range(0, n - i - 1):
            # Compare adjacent elements
            if scores[j] > scores[j + 1]:
                # Swap if the element found is greater than the next element
                scores[j], scores[j + 1] = scores[j + 1], scores[j]
                swapped = True

        # If no two elements were swapped by inner loop, then the list is sorted
        if not swapped:
            break
    return scores

# Sample student exam scores
student_scores = [85, 72, 93, 68, 72, 99, 81, 75]

print("Original scores:", student_scores)

# Sort the scores using Bubble Sort
sorted_scores = bubble_sort(student_scores)

print("Sorted scores:", sorted_scores)
```

```
print("Sorted scores:", sorted_scores)
```

```
Original scores: [85, 72, 93, 68, 72, 99, 81, 75]
Sorted scores: [68, 72, 72, 75, 81, 85, 93, 99]
```

Explanation : $O(n)$ Occurs when the array is already sorted. The algorithm makes one pass, detects no swaps, and terminates early due to the swapped flag. It performs $n-1$ comparisons. Average-Case Complexity: $O(n^2)$ For a randomly ordered array, on

average, the algorithm still performs approximately $n^2/2$ comparisons and $n^2/4$ swaps.

Task 2: Improving Sorting for Nearly Sorted Attendance Records

Scenario

You are maintaining an **attendance system** where student roll numbers are already *almost sorted*, with only a few late updates.

Task Description

- Start with a Bubble Sort implementation.
- Ask AI to:
 - Review the problem and suggest a more suitable sorting algorithm
 - Generate an **Insertion Sort** implementation
 - Explain why Insertion Sort performs better on nearly sorted data
- Compare execution behavior on nearly sorted input

Expected Outcome

- Two sorting implementations:
 - Bubble Sort
 - Insertion Sort
- AI-assisted explanation highlighting efficiency differences for partially sorted datasets

Prompt :

Implement Bubble Sort in Python to sort nearly sorted student roll numbers in an attendance system, then ask AI to review and suggest a more suitable algorithm. Generate an Insertion Sort implementation and explain why it performs better on nearly sorted data. Compare execution behavior of Bubble Sort and Insertion Sort on partially sorted input, highlighting efficiency differences

Code :

```
nearly_sorted_roll_numbers = list(range(1, 21))

# Introduce two elements out of their sorted positions
# Swap element at index 3 with element at index 8
nearly_sorted_roll_numbers[3], nearly_sorted_roll_numbers[8] = nearly_sorted_roll_numbers[8], nearly_sorted_roll_numbers[3]

# Swap element at index 12 with element at index 17
nearly_sorted_roll_numbers[12], nearly_sorted_roll_numbers[17] = nearly_sorted_roll_numbers[17], nearly_sorted_roll_numbers[12]

print("Nearly Sorted Roll Numbers:", nearly_sorted_roll_numbers)
```

• Nearly Sorted Roll Numbers: [1, 2, 3, 9, 5, 6, 7, 8, 4, 10, 11, 12, 18, 14, 15, 16, 17, 13, 19, 20]

```
bubble_sorted_roll_numbers = bubble_sort(list(nearly_sorted_roll_numbers))
print("Bubble Sorted Roll Numbers:", bubble_sorted_roll_numbers)
```

• Bubble Sorted Roll Numbers: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]

```
def insertion_sort(arr):
    # Traverse through 1 to len(arr)
    for i in range(1, len(arr)):
        key = arr[i] # Store the current element as key

        # Move elements of arr[0..i-1], that are greater than key,
        # to one position ahead of their current position
        j = i - 1
        while j >= 0 and key < arr[j]:
            arr[j + 1] = arr[j] # Shift element to the right
            j -= 1 # Move to the left to compare with the next element
        arr[j + 1] = key # Place key at its correct position
    return arr

print("Insertion sort function defined.")
```

• Insertion sort function defined.

```
insertion_sorted_roll_numbers = insertion_sort(list(nearly_sorted_roll_numbers))
print("Insertion Sorted Roll Numbers:", insertion_sorted_roll_numbers)
```

• Insertion Sorted Roll Numbers: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]

Explanation : A nearly sorted list of 20 student roll numbers was successfully generated, with two pairs of elements intentionally displaced. Both Bubble Sort and Insertion Sort were successfully applied to the nearly sorted data, and both algorithms correctly sorted the list into ascending order: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]. Insertion Sort's efficiency on nearly sorted data: Its adaptive nature allows it to achieve performance close to $O(n)$ because elements are already close to their final positions, resulting in minimal shifts and comparisons (the inner while loop terminates early).

Task 3: Searching Student Records in a Database

Scenario

You are developing a **student information portal** where users search for student records by roll number.

Task Description

- Implement:
 - **Linear Search** for unsorted student data
 - **Binary Search** for sorted student data
- Use AI to:
 - Add docstrings explaining parameters and return values
 - Explain when Binary Search is applicable
 - Highlight performance differences between the two searches

Expected Outcome

- Two working search implementations with docstrings
- AI-generated explanation of:
 - Time complexity
 - Use cases for Linear vs Binary Search
- A short student observation comparing results on sorted vs unsorted lists

Prompt :

Implement Linear Search in Python for unsorted student records and Binary Search for sorted records in a student information portal. Add docstrings explaining parameters and return values. Use AI to explain when Binary Search is applicable and highlight performance differences between the two searches. Provide a short analysis of time complexity and a student observation comparing results on sorted vs unsorted lists.

Code :

```
[6] student_records = ['Alice', 'Charlie', 'Bob', 'David', 'Eve', 'Frank', 'Grace', 'Heidi', 'Ivan', 'Judy']  
/ Os print("Unsorted Student Records:", student_records)  
✓ Unsorted Student Records: ['Alice', 'Charlie', 'Bob', 'David', 'Eve', 'Frank', 'Grace', 'Heidi', 'Ivan', 'Judy']
```

```
▶ def linear_search(data_list, target):  
    """  
    Performs a linear search to find the target element in the given list.  
  
    Linear search sequentially checks each element of the list until a match  
    is found or the whole list has been searched.  
  
    Args:  
        data_list (list): The list of elements to be searched through.  
        target (any): The element to search for in the list.  
  
    Returns:  
        int: The index of the target element if found. Returns -1 if the  
             target element is not present in the list.  
    """  
    for index, element in enumerate(data_list):  
        if element == target:  
            return index  
    return -1  
  
print("Linear search function defined.")  
  
... Linear search function defined.
```

```
▶ student_records = ['Alice', 'Charlie', 'Bob', 'David', 'Eve', 'Frank', 'Grace', 'Heidi', 'Ivan', 'Judy']  
  
# Existing student  
target_existing = 'David'  
index_existing = linear_search(student_records, target_existing)  
  
if index_existing != -1:  
    print(f'{target_existing} found at index: {index_existing}')  
else:  
    print(f'{target_existing} not found in the records.')  
  
# Non-existing student  
target_non_existing = 'Zoe'  
index_non_existing = linear_search(student_records, target_non_existing)  
  
if index_non_existing != -1:  
    print(f'{target_non_existing} found at index: {index_non_existing}')  
else:  
    print(f'{target_non_existing} not found in the records.')  
  
... 'David' found at index: 3  
    'Zoe' not found in the records.
```

```

def binary_search(data_list, target):
    """
    Performs a binary search to find the target element in a sorted list.

    Binary search works by repeatedly dividing in half the portion of the list
    that could contain the target, until the target is found or the possible
    range is empty.

    Args:
        data_list (list): The sorted list of elements to be searched through.
        target (any): The element to search for in the list.

    Returns:
        int: The index of the target element if found. Returns -1 if the
            target element is not present in the list.
    """
    low = 0
    high = len(data_list) - 1

    while low <= high:
        mid = (low + high) // 2
        mid_element = data_list[mid]

        if mid_element == target:
            return mid # Target found
        elif mid_element < target:
            low = mid + 1 # Target is in the right half
        else:
            high = mid - 1 # Target is in the left half

```

```

        else:
            high = mid - 1 # Target is in the left half

    return -1 # Target not found in the list

print("Binary search function defined.")

```

Binary search function defined.

Explanantion : hile both algorithms correctly find existing elements and identify non-existing ones, Binary Search offers a significant performance advantage for large, sorted datasets due to its divide-and-conquer strategy, requiring far fewer comparisons than Linear Search. However, the requirement for sorted data is its primary limitation.

Task 4: Choosing Between Quick Sort and Merge Sort for Data Processing

Scenario

You are part of a **data analytics team** that needs to sort large datasets received from different sources (random order, already sorted, and reverse sorted).

Task Description

- Provide AI with partially written recursive functions for:

- Quick Sort
- Merge Sort
- Ask AI to:
 - Complete the recursive logic
 - Add meaningful docstrings
 - Explain how recursion works in each algorithm
- Test both algorithms on:
 - Random data
 - Sorted data
 - Reverse-sorted data

Expected Outcome

- Fully functional Quick Sort and Merge Sort implementations
- AI-generated comparison covering:
 - Best, average, and worst-case complexities
 - Practical scenarios where one algorithm is preferred over the other
 - Prompt :

Provide partially written recursive functions for Quick Sort and Merge Sort in Python, then ask AI to complete the recursive logic and add meaningful docstrings. Request an explanation of how recursion works in each algorithm. Test both implementations on random, sorted, and reverse-sorted datasets. Finally, generate an AI-assisted comparison of best, average, and worst-case complexities, along with practical scenarios where Quick Sort or Merge Sort is preferred.

Code :

```
def quick_sort(arr, low, high):
    """
    Partially implements the Quick Sort algorithm recursively.
    """
    if low < high:
        # Nested helper function for partitioning
        def partition(arr, low, high):
            pivot = arr[high] # Select the last element as the pivot
            i = low - 1 # Initialize pointer for greater element

            # Traverse through all elements, compare each with pivot
            for j in range(low, high):
                if arr[j] <= pivot:
                    i += 1 # If element smaller than pivot is found, swap it with the greater element pointed by i
                    arr[i], arr[j] = arr[j], arr[i]

            # Swap the pivot element with the element at i + 1
            arr[i + 1], arr[high] = arr[high], arr[i + 1]
            return i + 1 # Return the partitioning index

        pi = partition(arr, low, high)

        # Recursive calls for elements before and after partition
        # These recursive calls will be completed in the next step
        # quick_sort(arr, low, pi - 1)
        # quick_sort(arr, pi + 1, high)
```



```
# Recursive calls for elements before and after partition
# These recursive calls will be completed in the next step
# quick_sort(arr, low, pi - 1)
# quick_sort(arr, pi + 1, high)
```

```
print("Quick Sort function (partially) defined.")
```

Quick Sort function (partially) defined.

```
def quick_sort(arr, low, high):
    """
    Implements the Quick Sort algorithm recursively to sort a list in-place.

    Quick Sort is a divide-and-conquer algorithm that works by selecting a 'pivot'
    element from the array and partitioning the other elements into two sub-arrays,
    according to whether they are less than or greater than the pivot. The sub-arrays
    are then sorted recursively. This process is repeated until the entire list is sorted.

    Args:
        arr (list): The list of elements to be sorted. This list will be modified in-place.
        low (int): The starting index of the sub-array to be sorted.
        high (int): The ending index of the sub-array to be sorted.

    Returns:
        None: The function sorts the list in-place and does not return a new list.
    """
    if low < high:
        # Nested helper function for partitioning
        def partition(arr, low, high):
            pivot = arr[high] # Select the last element as the pivot
            i = low - 1 # Initialize pointer for greater element

            # Traverse through all elements, compare each with pivot
            for j in range(low, high):
                if arr[j] <= pivot:
                    i += 1 # If element smaller than pivot is found, swap it with the greater element pointed by i
                    arr[i], arr[j] = arr[j], arr[i]

            # Swap the pivot element with the element at i + 1
            arr[i + 1], arr[high] = arr[high], arr[i + 1]
            return i + 1 # Return the partitioning index

        pi = partition(arr, low, high)

        # Recursive calls for elements before and after partition
        quick_sort(arr, low, pi - 1)
        quick_sort(arr, pi + 1, high)

    print("Quick Sort function fully defined.")
```

Quick Sort function fully defined.

```

14]
✓ Os
def merge_sort(arr):
    """
    Partially implements the Merge Sort algorithm recursively.
    """
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left_half = arr[:mid]
    right_half = arr[mid:]

    # Recursive calls for sorting left and right halves
    # These recursive calls will be completed in the next step
    # left_half = merge_sort(left_half)
    # right_half = merge_sort(right_half)

    # This is where the merge logic would go to combine the two sorted halves
    # For now, we return the original array as the merge logic is not yet implemented
    return arr

print("Merge Sort function (partially) defined.")

```

... Merge Sort function (partially) defined.

```

def merge_sort(arr):
    """
    Implements the Merge Sort algorithm recursively to sort a list.

    Merge Sort is a divide-and-conquer algorithm that divides the unsorted list
    into n sublists, each containing one element (a list of one element is
    considered sorted). Then, it repeatedly merges sublists to produce new sorted
    sublists until there is only one sorted list remaining.

    Args:
        arr (list): The list of elements to be sorted.

    Returns:
        list: A new sorted list containing all elements from the original list.
    """
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left_half = arr[:mid]
    right_half = arr[mid:]

    # Recursive calls for sorting left and right halves
    left_half = merge_sort(left_half)
    right_half = merge_sort(right_half)

    # Helper function to merge two sorted lists

```

```

while i < len(left) and j < len(right):
    if left[i] <= right[j]:
        merged.append(left[i])
        i += 1
    else:
        merged.append(right[j])
        j += 1

# Append any remaining elements from left_half
while i < len(left):
    merged.append(left[i])
    i += 1

# Append any remaining elements from right_half
while j < len(right):
    merged.append(right[j])
    j += 1
return merged

# Merge the sorted halves
return merge(left_half, right_half)

print("Merge Sort function fully defined.")

```

... Merge Sort function fully defined.

```

import random

# 1. Create a list called random_data containing 10 random integers between 1 and 100 (inclusive).
random_data = [random.randint(1, 100) for _ in range(10)]

# 2. Create a sorted version of random_data called sorted_data.
sorted_data = sorted(random_data)

# 3. Create a reverse-sorted version of random_data called reverse_sorted_data.
reverse_sorted_data = sorted(random_data, reverse=True)

# 4. Print all three lists: random_data, sorted_data, and reverse_sorted_data.
print("Random Data:", random_data)
print("Sorted Data:", sorted_data)
print("Reverse Sorted Data:", reverse_sorted_data)

```

... Random Data: [83, 8, 1, 58, 72, 74, 44, 25, 65, 40]
Sorted Data: [1, 8, 25, 40, 44, 58, 65, 72, 74, 83]
Reverse Sorted Data: [83, 74, 72, 65, 58, 44, 40, 25, 8, 1]

Explanation : Both Quick Sort and Merge Sort are efficient comparison-based sorting algorithms with an average time complexity of $O(n \log n)$. **Quick Sort** is often faster in practice due to a smaller constant factor in its average-case $O(n \log n)$ performance and its in-place nature (better cache performance).

Task 5: Optimizing a Duplicate Detection Algorithm

Scenario

You are building a **data validation module** that must detect duplicate user IDs in a large dataset before importing it into a system.

Task Description

- Write a **naive duplicate detection algorithm** using nested loops.
- Use AI to:
 - Analyze the time complexity
 - Suggest an optimized approach using sets or dictionaries
 - Rewrite the algorithm with improved efficiency
- Compare execution behavior conceptually for large input sizes

Expected Outcome

- Two versions of the algorithm:
 - Brute-force ($O(n^2)$)
 - Optimized ($O(n)$)
 - AI-assisted explanation showing how and why performance improved
- Prompt :

Write a naive duplicate detection algorithm in Python using nested loops to check for duplicate user IDs in a dataset. Ask AI to analyze its time complexity, suggest an optimized approach using sets or dictionaries, and rewrite the algorithm with improved efficiency. Compare execution behavior conceptually for large input sizes, showing both the brute-force $O(n^2)$ version and the optimized $O(n)$ version, with an explanation of how and why performance improved

Code :

```
def find_duplicates_naive(user_ids):  
    """  
    Detects duplicate user IDs in a list using a naive approach with nested loops.  
  
    This function compares each user ID with every other user ID to find duplicates.  
    It immediately returns True if any duplicate pair is found.  
  
    Args:  
        user_ids (list): A list of user IDs (can contain any comparable elements).  
  
    Returns:  
        bool: True if duplicate user IDs are found, False otherwise.  
    """  
    n = len(user_ids)  
    for i in range(n):  
        for j in range(n):  
            # Ensure an element is not compared with itself  
            if i != j and user_ids[i] == user_ids[j]:  
                return True # Duplicate found  
        return False # No duplicates found after checking all pairs  
  
    # Sample list with duplicates  
    sample_with_duplicates = [1, 2, 3, 2, 4]  
    print(f"Does {sample_with_duplicates} contain duplicates? {find_duplicates_naive(sample_with_duplicates)}")  
  
    # Sample list with no duplicates  
    sample_no_duplicates = [1, 2, 3, 4, 5]  
    print(f"Does {sample_no_duplicates} contain duplicates? {find_duplicates_naive(sample_no_duplicates)}")
```

```
sample_no_duplicates = [1, 2, 3, 4, 5]  
print(f"Does {sample_no_duplicates} contain duplicates? {find_duplicates_naive(sample_no_duplicates)}")  
  
Does [1, 2, 3, 2, 4] contain duplicates? True  
Does [1, 2, 3, 4, 5] contain duplicates? False
```

Explanation : The `find_duplicates_naive` function, which uses nested loops, has a significant inefficiency. It compares each element with every other element in the list. This leads to $n * (n-1)$ comparisons in the worst case (or $n*(n-1)/2$ if avoiding self-comparison and redundant pairs), resulting in a time complexity of $O(n^2)$. For small lists, this might be acceptable, but as the size of the list (n) grows, the number of comparisons increases quadratically, making it very slow and inefficient.